

The logo for Oracle Academy. The word "ORACLE" is in a bold, orange, sans-serif font. Below it, the word "Academy" is in a smaller, dark gray, sans-serif font. The entire logo is centered on a light gray background, which is framed by dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Programming with PL/SQL

12-1

Using Dynamic SQL

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Recall the stages through which all SQL statements pass
 - Describe the reasons for using dynamic SQL to create a SQL statement
 - List four PL/SQL statements supporting Native Dynamic SQL
 - Describe the benefits of EXECUTE IMMEDIATE over DBMS_SQL for Dynamic SQL

Purpose

- In this lesson, you learn to construct and execute SQL statements dynamically—in other words, at run time using the Native Dynamic SQL statements in PL/SQL
- Dynamically executing SQL and PL/SQL code extends the capabilities of PL/SQL beyond query and transactional operations
- The lesson also compares Native Dynamic SQL to the DBMS_SQL package, which provides similar capabilities

Execution Flow of SQL

- All SQL statements in the database go through various stages:
 - Parse: Pre-execution “is this possible?” checks syntax, object existence, privileges, and so on
 - Bind: Getting the actual values of any variables referenced in the statement
 - Execute: The statement is executed
 - Fetch: Results are returned to the user
- Some stages might not be relevant for all statements; for example, the fetch phase is applicable to queries but not DML

SQL statements operate independently. When writing programs you must create a body of code that is going to vary according to the data and the user input.

PL/SQL is an application-development language that is a superset of SQL, supplementing it with standard programming-language features and PL/SQL allows you to store compiled code directly in the database.

Execution Flow of SQL in PL/SQL Subprograms

- When a SQL statement is included in a PL/SQL subprogram, the parse and bind phases are normally done at compile time, that is, when the procedure, function, or package body is CREATED
- What if the text of the SQL statement is not known when the procedure is created?



Execution Flow of SQL in PL/SQL Subprograms

- How could the Oracle server parse it?
- It couldn't
- For example, suppose you want to DROP a table, but the user enters the table name at execution time:

```
CREATE PROCEDURE drop_any_table(p_table_name VARCHAR2)
IS BEGIN
  DROP TABLE p_table_name; -- cannot be parsed
END;
```

Dynamic SQL

- The embedded SQL statements available in PL/SQL are limited to :
 - SELECT, INSERT, UPDATE, DELETE, COMMIT, and ROLLBACK,
 - All of which are parsed at compile time; that is, they have a fixed structure
- You need to use Dynamic SQL functionality if you require:
 - The structure of a SQL statement to be altered at execution time
 - Access to DDL statements and other SQL functionality in PL/SQL

Dynamic SQL

- You use dynamic SQL to create a SQL statement whose text is not completely known in advance
- Dynamic SQL:
 - Is constructed and stored as a character string within a subprogram
 - Is a SQL statement with varying column data, or different conditions with or without placeholders (bind variables)
 - Enables data-definition, data-control, or session-control statements to be written and executed from PL/SQL

Native Dynamic SQL

- PL/SQL does not support DDL statements written directly in a program
- Native Dynamic SQL (NDS) allows you to work around this by constructing and storing SQL as a character string within a subprogram
- NDS:
 - Provides native support for Dynamic SQL directly in the PL/SQL language
 - Enables data-definition, data-control, or session-control statements to be written and executed from PL/SQL

In Oracle 8 and earlier releases, the only way to implement Dynamic SQL in a PL/SQL application was by using the DBMS_SQL package. With Oracle 8i and later releases, the PL/SQL environment provides NDS as an easier alternative.

Native Dynamic SQL

- NDS:
 - Is executed with Native Dynamic SQL statements (EXECUTE IMMEDIATE) or the DBMS_SQL package
 - Provides the ability to execute SQL statements whose structure is unknown until execution time
 - Can also use the OPEN-FOR, FETCH, and CLOSE PL/SQL statements



Using the EXECUTE IMMEDIATE Statement

- Use the EXECUTE IMMEDIATE statement for NDS in PL/SQL anonymous blocks or subprograms:

```
EXECUTE IMMEDIATE dynamic_string  
  [INTO {define_variable  
    [, define_variable] ... | record}]  
  [USING [IN|OUT|IN OUT] bind_argument  
    [, [IN|OUT|IN OUT] bind_argument] ... ];
```

- INTO is used for single-row queries and specifies the variables or records into which column values are retrieved
- USING holds all bind arguments
- The default parameter mode is IN, if not specified

Using the EXECUTE IMMEDIATE Statement

```
EXECUTE IMMEDIATE dynamic_string
  [INTO {define_variable
    [, define_variable] ... | record}]
  [USING [IN|OUT|IN OUT] bind_argument
    [, [IN|OUT|IN OUT] bind_argument] ... ];
```

- *dynamic_string* is a character variable or literal containing the text of a SQL statement
- *define_variable* is a PL/SQL variable that stores a selected column value
- *record* is a user-defined or %ROWTYPE record that stores a selected row

You can use numeric, character, and date literals as bind arguments, but you cannot use Boolean literals (TRUE, FALSE, and NULL).

Using the EXECUTE IMMEDIATE Statement

```
EXECUTE IMMEDIATE dynamic_string  
  [INTO {define_variable  
    [, define_variable] ... | record}]  
  [USING [IN|OUT|IN OUT] bind_argument  
    [, [IN|OUT|IN OUT] bind_argument] ... ];
```

- *bind_argument* is an expression whose value is passed to the dynamic SQL statement at execution time
- USING clause holds all bind arguments
- The default parameter mode is IN

Example 1: Dynamic SQL with a DDL Statement

- Constructing the dynamic statement in-line:

```
CREATE PROCEDURE drop_any_table(p_table_name VARCHAR2) IS
BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE ' || p_table_name;
END;
```

- Constructing the dynamic statement in a variable:

```
CREATE PROCEDURE drop_any_table(p_table_name VARCHAR2) IS
    v_dynamic_stmt VARCHAR2(50);
BEGIN
    v_dynamic_stmt := 'DROP TABLE ' || p_table_name;
    EXECUTE IMMEDIATE v_dynamic_stmt;
END;
```

```
BEGIN drop_any_table('EMPLOYEE_NAMES'); END;
```

The two procedures in the slide are functionally identical. Note the first example, using EXECUTE IMMEDIATE, is much simpler and easier to read.

This and the following examples show Dynamic SQL within standalone procedures and functions, but Dynamic SQL can be used within packaged subprograms in the same way.

Example 2: Dynamic SQL with a DML Statement

- Deleting all the rows from any table and returning a count:

```
CREATE FUNCTION del_rows(p_table_name VARCHAR2)
RETURN NUMBER IS
BEGIN
    EXECUTE IMMEDIATE 'DELETE FROM ' || p_table_name;
    RETURN SQL%ROWCOUNT;
END;
```

- Invoking the function:

```
DECLARE
    v_count NUMBER;
BEGIN
    v_count := del_rows('EMPLOYEE_NAMES');
    DBMS_OUTPUT.PUT_LINE(v_count || ' rows deleted.');
```

```
END;
```


Example 3: Dynamic SQL with a DML Statement

- Here is an example of inserting a row into a table with two columns and invoking the procedure
- Note the use of escape single quotes

```
CREATE PROCEDURE add_row(p_table_name VARCHAR2,  
    p_id NUMBER, p_name VARCHAR2) IS  
BEGIN  
    EXECUTE IMMEDIATE 'INSERT INTO ' || p_table_name ||  
        'VALUES(' || p_id || ', ' || p_name || ')';  
END;
```

```
BEGIN  
    add_row('EMPLOYEE_NAMES', 250, 'Chang');  
END;
```

ORACLE
Academy

PLSQL 12-1
Using Dynamic SQL

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

17

When this procedure runs with the actual parameters indicated in the example above, the EXECUTE IMMEDIATE (and the escape single quotes) will result in this INSERT statement:

```
INSERT INTO employee_names VALUES(250, 'Chang')
```

The two uses of escape single quotes results in the single quote before and after Chang in the resulting INSERT statement. A single quote followed immediately by another single quote, is an example of using an escape character to indicate that the, in the case, single quote, should be treated as a string character rather than as a syntactical token. The following, third single quote is unrelated to the two previous single quotes, and it is the syntactical token that ends the string.

Example 4: Using Native Dynamic SQL to Recompile PL/SQL Code

- You can recompile PL/SQL objects without recreating them by using the following ALTER statements:

```
ALTER PROCEDURE procedure-name COMPILE;  
ALTER FUNCTION  function-name  COMPILE;  
ALTER PACKAGE package_name COMPILE SPECIFICATION;  
ALTER PACKAGE package-name COMPILE BODY;
```

- If you leave out the keyword SPECIFICATION or BODY with the ALTER PACKAGE statement, then the specification and body are both compiled

Example 4: Using Native Dynamic SQL to Recompile PL/SQL Code

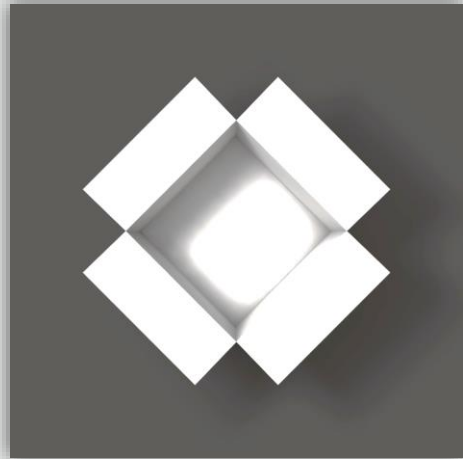
- This example creates a procedure that recompiles a PL/SQL object whose name and type is entered at run time

```
CREATE PROCEDURE compile_plsql
  (p_name VARCHAR2,p_type VARCHAR2,p_options VARCHAR2 := NULL)
IS
  v_stmt VARCHAR2(200);
BEGIN
  v_stmt := 'ALTER ' || p_type || ' ' || p_name || ' COMPILE'
    || ' ' || p_options;
  EXECUTE IMMEDIATE v_stmt;
END;

BEGIN compile_plsql('MYPACK','PACKAGE','BODY'); END;
```

Using the DBMS_SQL Package

- Some of the procedures and functions of the DBMS_SQL package are:
 - OPEN_CURSOR
 - PARSE
 - BIND_VARIABLE
 - EXECUTE
 - FETCH_ROWS
 - CLOSE_CURSOR



Before Oracle 8i, the EXECUTE IMMEDIATE statement did not exist in PL/SQL, and the presupplied DBMS_SQL package was the only way to write Dynamic SQL.

The purpose of this and the examples in the next two slides is to show that DBMS_SQL has no advantages over EXECUTE IMMEDIATE and is much more long-winded and difficult to use. Using EXECUTE IMMEDIATE is therefore strongly recommended.

Using DBMS_SQL with a DML Statement

- Example of deleting rows:

```
CREATE OR REPLACE FUNCTION del_rows
(p_table_name VARCHAR2) RETURN NUMBER IS
  v_csr_id INTEGER;
  v_rows_del NUMBER;
BEGIN
  v_csr_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(v_csr_id,
    'DELETE FROM ' || p_table_name, DBMS_SQL.NATIVE);
  v_rows_del := DBMS_SQL.EXECUTE(v_csr_id);
  DBMS_SQL.CLOSE_CURSOR(v_csr_id);
  RETURN v_rows_del;
END;
```

- Compare this with the del_rows function earlier in this lesson
- They are functionally identical, but which is simpler?

ORACLE
Academy

PLSQL 12-1
Using Dynamic SQL

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

21

The calls to DBMS_SQL perform the following actions:

Use DBMS_SQL.OPEN_CURSOR to establish a cursor (area in memory) to process a SQL statement.

Use DBMS_SQL.PARSE to establish the validity of the SQL statement.

Use DBMS_SQL.EXECUTE function to run the SQL statement. This function returns the number of rows processed.

Use DBMS_SQL.CLOSE_CURSOR to close the cursor.

Using DBMS_SQL with a Parameterized DML Statement

- Again, compare this with the add_row procedure earlier in this lesson
- Which would you rather write?

```
CREATE PROCEDURE add_row (p_table_name VARCHAR2,  
    p_id NUMBER, p_name VARCHAR2) IS  
    v_csr_id    INTEGER;  
    v_stmt     VARCHAR2(200);  
    v_rows_added NUMBER;  
BEGIN  
    v_stmt := 'INSERT INTO ' || p_table_name ||  
        ' VALUES(' || p_id || ', ' || p_name || ')';  
    v_csr_id := DBMS_SQL.OPEN_CURSOR;  
    DBMS_SQL.PARSE(v_csr_id, v_stmt, DBMS_SQL.NATIVE);  
    v_rows_added := DBMS_SQL.EXECUTE(v_csr_id);  
    DBMS_SQL.CLOSE_CURSOR(v_csr_id);  
END;
```

ORACLE
Academy

PLSQL 12-1
Using Dynamic SQL

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

22

You don't have to understand everything shown here to understand that using EXECUTE IMMEDIATE is much simpler, and therefore recommended.

Comparison of Native Dynamic SQL and the DBMS_SQL Package

- Native Dynamic SQL:
 - Is easier to use than DBMS_SQL
 - Requires less code than DBMS_SQL
 - Often executes faster than DBMS_SQL because there are fewer statements to execute



Terminology

- Key terms used in this lesson included:
 - Native Dynamic SQL
 - EXECUTE IMMEDIATE

- Native Dynamic SQL – some programs must build and process a variety of SQL statements at run time. Such statements can, and probably will, change from execution to execution.
- EXECUTE IMMEDIATE – A statement that prepares (parses) and immediately executes a dynamic SQL statement or an anonymous PL/SQL block.

Summary

- In this lesson, you should have learned how to:
 - Recall the stages through which all SQL statements pass
 - Describe the reasons for using dynamic SQL to create a SQL statement
 - List four PL/SQL statements supporting Native Dynamic SQL
 - Describe the benefits of EXECUTE IMMEDIATE over DBMS_SQL for Dynamic SQL

The Oracle Academy logo is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by two horizontal dark gray bars, one at the top and one at the bottom.

ORACLE

Academy